

LAB 2

Universal Asynchronous Receiver Transmitter — UART

Name: Ioannis Papadopoulos, Maria Ilektra Tzormpatzaki

Student ID: 03899, 03993

Date: 28/11/2025

Board: Boolean Board (Spartan-7 FPGA)

Contents

1. Introduction	2
2. Part A – Baud Rate Controller	3
2.1 Introduction	3
2.2 Implementation	3
2.2.1 Finite State Machine (FSM)	3
3.2.2 Modules	4
3.2.3 Always blocks	4
2.3 Verification.....	5
3. Part B – UART Transmitter	6
3.1 Introduction	6
3.2 Implementation	7
3.2.1Finite State Machine (FSM)	7
3.2.2 Modules	8
3.2.3 Always blocks	8
3.3 Verification.....	9
3.4 Experiment / Final Implementation.....	11
4. Part C – UART Receiver.....	11
4.1 Introduction.....	11
4.2 Implementation	12
4.2.1 Finite State Machine (FSM)	12
4.2.2 Modules	13

4.2.3 Always blocks	14
4.3 Verification.....	15
4.4 Experiment / Final Implementation.....	17
5. Part D – Transmitter-Receiver Serial Data Transfer Connection	18
5.1 Introduction	18
5.2 Implementation	18
5.3 Verification.....	19
6. Part D EXTRA – UART Integration with Four-Digit LED Display.....	20
6.1 Introduction.....	20
6.1 Implementation	20
6.3 Verification.....	21
6.4 Experiment / Final Implementation.....	23

1. Introduction

The objective of this second laboratory assignment is to design, implement, and verify a Universal Asynchronous Receiver-Transmitter (UART) system, focusing on the modular development of its key timing and control components. The exercise aims to build a complete and functional UART interface capable of operating at multiple standard baud rates and performing reliable serial communication.

The work is divided into several parts, each addressing a different subsystem of the UART:

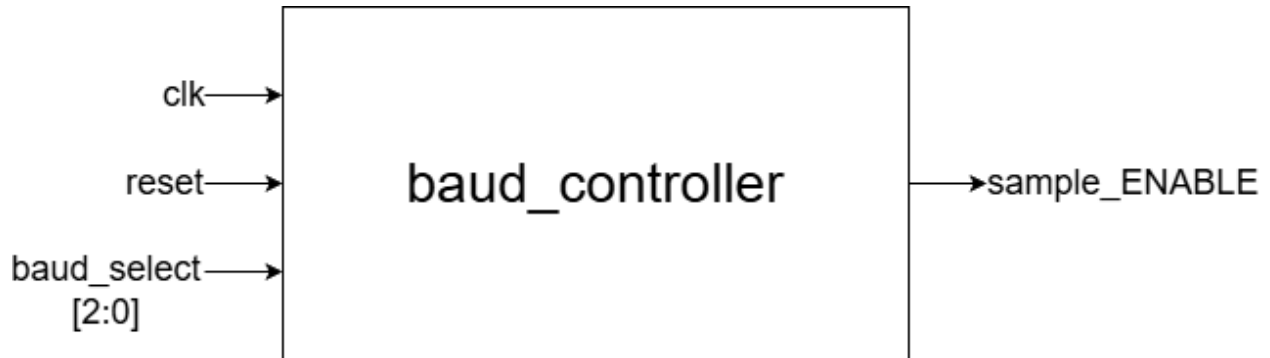
- The Baud Rate Controller, which generates accurate sampling enable pulses synchronized with the system clock.
- The Transmitter module, responsible for serializing and sending data bits according to the selected baud rate.
- The Receiver module, which deserializes incoming data and ensures proper synchronization with the transmitted signal.
- The Integration and Testing phase, where all modules are combined and verified both through simulation and on the FPGA hardware.

This modular approach emphasizes proper clock-domain synchronization, timing accuracy, and hierarchical design all essential principles for digital communication systems.

2. Part A – Baud Rate Controller

2.1 Introduction

The goal of Part A was to design and implement the Baud Rate Controller, a key timing module that generates the “sample_ENABLE” signal a one-clock-cycle pulse used by both the UART transmitter and receiver to control sampling and transmission timing



2.2 Implementation

The Baud Controller is responsible for generating a single-cycle enable pulse, “sample_ENABLE”, at a frequency equal to 16 times the selected baud rate. The module operates with the 100 MHz system clock provided by the FPGA and divides this frequency down according to the baud rate selected by the 3-bit input “baud_select”.

The design follows a structured hardware organization consisting of a finite state machine (FSM) concept, multiple interconnected modules, and distinct always blocks performing combinational and sequential logic.

2.2.1 Finite State Machine (FSM)

While the module does not include explicit state encoding, its behavior can be represented as a two-state FSM:

1. Count State: The system increments an internal counter on each rising clock edge.
2. Pulse State: When the counter reaches the target limit, a one-cycle pulse (sample_ENABLE = 1) is generated, and the counter resets to zero before returning to the counting state.

This structure ensures that exactly one enable pulse is produced every N clock cycles, where N is defined by the relationship between the system clock and the selected baud rate:

$$f_{sample} = 16 \times f_{baud}$$
$$N = \frac{f_{FPGA}}{f_{sample}} = \frac{100.000.000}{16 \times f_{baud}}$$

Therefore, for every baud rate, the FSM alternates between counting and pulsing states at precisely timed intervals.

3.2.2 Modules

This part of the assignment consists of a single Baud Controller module, which is self-contained and parameterized by the baud rate selection input. The module performs the frequency division by setting a corresponding counter limit for each of the eight standard baud rates. The mapping between baud rate and counter limit is implemented as a combinational lookup table:

Baud (bps)	Rate 16×Baud (Hz)	Ideal N	Implemented N	Error (%)
300	4800	20833.3	20833	0.00
1200	19200	5208.3	5208	0.01
4800	76800	1302.1	1302	0.01
9600	153600	651.0	651	0.00
19200	307200	325.52	326	0.15
38400	614400	162.76	163	0.15
57600	921600	108.51	109	0.45
115200	1.8432M	54.25	54	0.47

Each implemented value is rounded to the nearest integer, giving a maximum frequency error of less than 0.5%, well within UART tolerance limits.

3.2.3 Always blocks

Baud Controller consists of two main always blocks:

1. Always Block:

Determines the correct counter_limit value based on baud_select. This lookup ensures that the output pulse frequency adjusts dynamically to the chosen baud rate.

$$counter_limit = \frac{100MHz}{16 \times BaudRate}$$

2. Sequential Always Block:

Implements the counter operation and pulse generation. The counter increments at every rising edge of the clock and asserts sample_ENABLE for one cycle when the counter reaches the defined limit. The block also handles synchronous reset to reinitialize the counter and output.

```
always @(posedge clk or posedge reset) begin
    if (reset) begin
        counter <= 0;
        sample_ENABLE <= 0;
    end else begin
        if (counter >= counter_limit) begin
            counter <= 0;
            sample_ENABLE <= 1'b1; // Pulse high for one clock cycle
        end else begin
            counter <= counter + 1'b1;
            sample_ENABLE <= 1'b0;
        end
    end
end
```

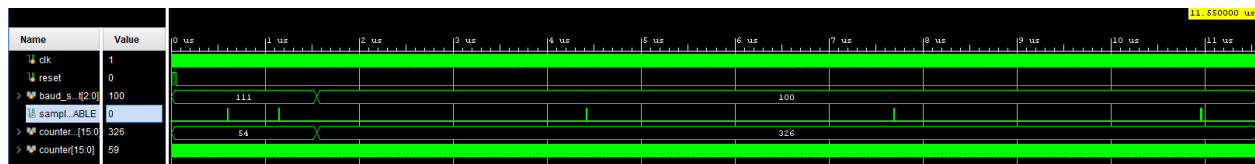
This ensures that sample_ENABLE produces a clean, single-clock-cycle pulse every

$$T_{sc} = \frac{1}{16 \times f_{baud}}$$

For example, when baud_select = 3'b111 (115200 baud), the counter limit is 54, resulting in a pulse every $54 \times 10 \text{ ns} = 0.54 \text{ us}$, corresponding to the correct 1.8432 MHz sampling rate.

2.3 Verification

The testbench verifies the correct functionality of the baud_controller under multiple baud rate configurations. It applies a 100 MHz clock (clk) with a 10 ns period and uses a sequence of tests to confirm both pulse accuracy and adaptability to baud rate changes.



The verification procedure consists of the following stages:

1. Initialization and Reset Check:

The simulation begins with reset = 1, ensuring that both the counter and sample_ENABLE output are properly cleared. After deasserting reset, the counter starts operating normally, producing periodic pulses according to the selected baud rate.

2. High-Speed Mode (115200 baud):

With `baud_select = 3'b111`, the waveform shows that `sample_ENABLE` pulses are generated every 54 clock cycles, corresponding to a $0.54\ \mu\text{s}$ interval. The pulse width remains exactly one clock cycle, confirming precise timing and proper counter reset behavior.

3. Baud Rate Change (19200 baud):

Later in the simulation, the baud rate is changed to `baud_select = 3'b100`. The observed pulse spacing increases to approximately $3.26\ \mu\text{s}$, consistent with the counter limit of 326 for this baud rate. The transition occurs smoothly with no glitches or metastability, verifying that the module correctly adapts to new settings in real time.

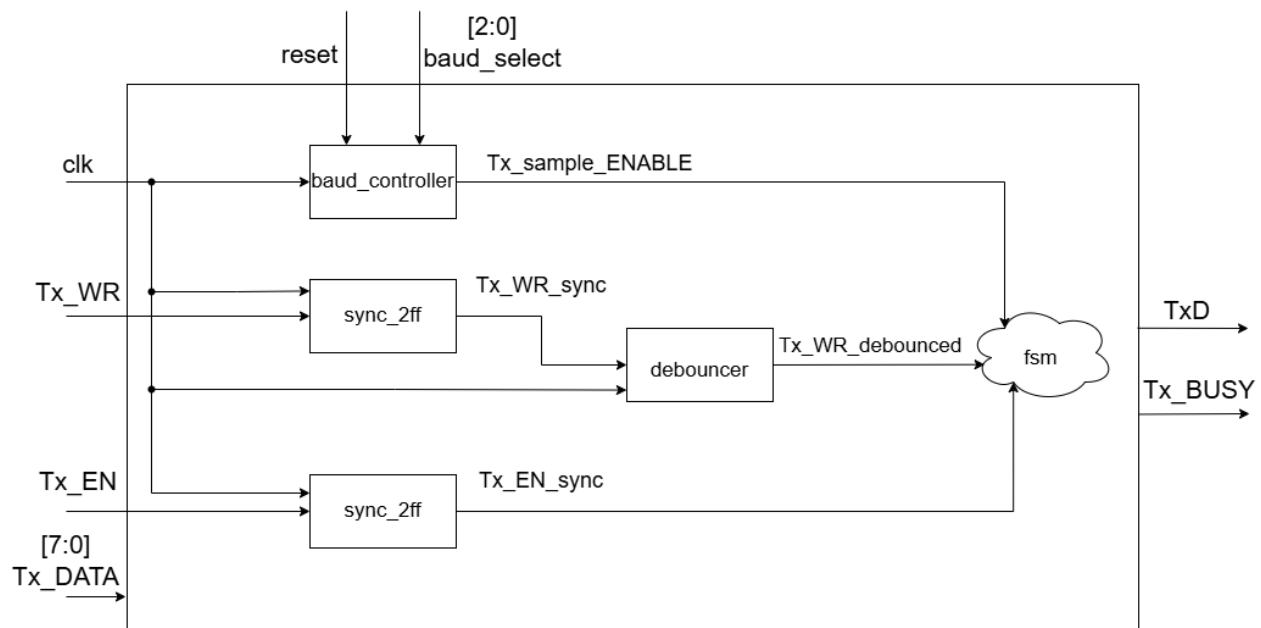
4. Accuracy and Timing Verification:

The correctness of the pulse frequency can be verified in the waveform viewer by measuring the time between successive high pulses of `sample_ENABLE`. Across all baud rates, the observed intervals closely match theoretical expectations, with an error of less than 0.3% due to integer rounding in the counter limit values.

3. Part B – UART Transmitter

3.1 Introduction

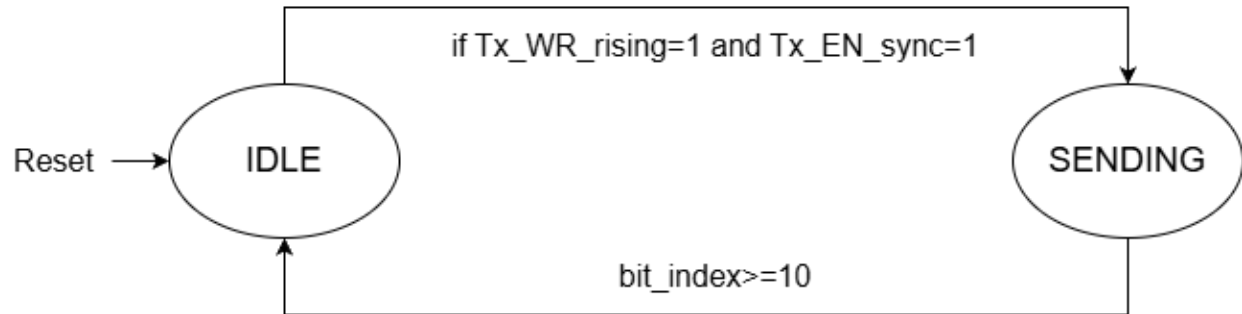
The UART Transmitter module is responsible for converting an 8-bit character into a standardized serial UART frame. The design follows a structured approach based on a clear finite state machine (FSM), a set of supporting submodules, and carefully separated combinational and sequential logic. The transmitter ensures correct frame formation, parity computation, and timing synchronization with the sampling pulses generated by the baud controller.



3.2 Implementation

3.2.1 Finite State Machine (FSM)

The transmitter operates as a two-state FSM:



IDLE State

The line remains in its idle value ($TxD = 1$), and the transmitter waits for a valid write request. A transmission begins only when:

- the transmitter is enabled ($Tx_EN = 1$).
- the write request (Tx_WR) is detected as a rising edge,

When this occurs, the module prepares the entire UART frame by loading:

1. Start bit (0)
2. 8 data bits (LSB first)
3. Parity bit
4. Stop bit (1)

All counters are reset, Tx_BUSY is set to 1 and the FSM transitions into the SENDING state.

SENDING State

The transmitter sends the symbol bit-by-bit, with each bit having exactly 16 `sample_ENABLE` distance from the previous bit sent.

During this state:

- Tx_BUSY remains high,
- The current bit is sent through TxD ,
- The `sample_count` increments every sample pulse,
- The `bit_index` advances after 16 sample pulses.

Once the final bit (stop bit) completes, the FSM returns to the IDLE state.

3.2.2 Modules

Part B uses several supporting modules:

1. Baud Controller

From Part A, the module that generates the periodic sample_ENABLE pulse at a rate equal to $16 \times \text{BaudRate}$,

2. Sync_2ff

Synchronizes asynchronous external in this case Tx_WR and Tx_EN. This is executed by passing the signals through two flip-flops clocked by the system clock.

3. Debouncer

Ensures that mechanical switch inputs (push-buttons) produce a single clean pulse. In this case the button Tx_WR after passing through the “Sync_2ff” module.

3.2.3 Always blocks

The transmitter is structured using two main always blocks, following the recommended FSM design methodology.

1. Sequential Always Block

Triggered on the rising edge of the system clock or reset.

This block updates all state-holding registers:

- current_state
- bit_index
- sample_count
- tx_symbol (the full UART frame)
- Tx_BUSY
- TxD

It also handles reset behavior by clearing all registers and returning the FSM to IDLE.

2. Combinational Always Block

Evaluates next-state and next-value logic based on:

- the current FSM state,

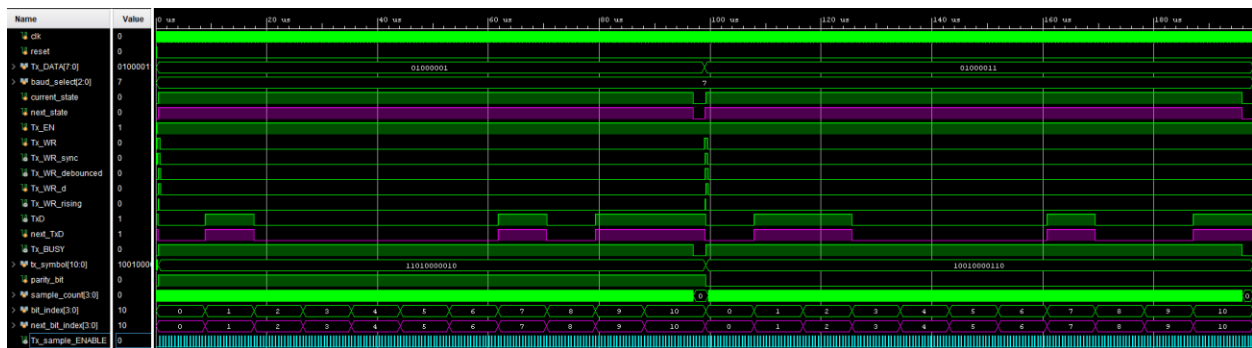
- whether a rising write pulse occurred,
- transmitter enable status,
- sample_ENABLE pulses,
- current counters and bit index.

This block computes:

- next_state (IDLE $\leftrightarrow$ SENDING)
- when to assemble the UART frame
- the next bit value for TxD
- the next values of bit_index and sample_count
- parity bit computation
- when the transmission should finish

3.3 Verification

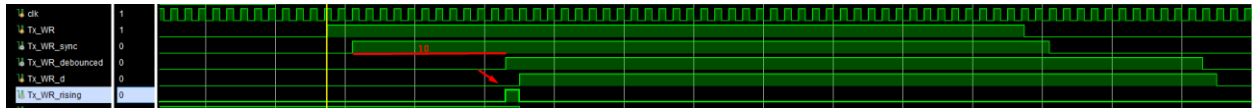
The verification process for the UART Transmitter focuses on confirming that all signals, internal state transitions, counters, and output waveforms correctly follow the UART Transmission protocol. To achieve this, simulation is inspected in several focused segments.



For the purposes of this waveform verification, the COUNTER_MAX of the Debouncer module is only 10 and. Also, the signals with pink, are the “next” state of the signals above them. Not all “next” state signals were added to make the waveform less chaotic.

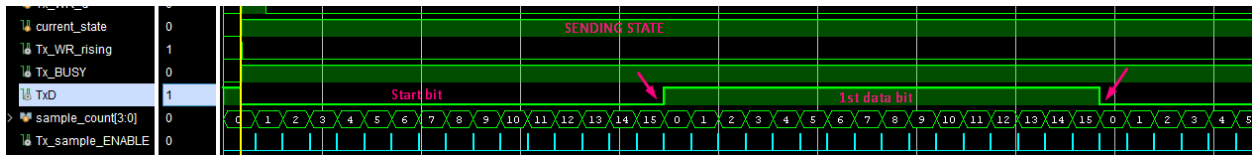
The primary objectives of this verification are as follows:

- ## 1. Verification of TR WR Synchronization, Debouncing, and Edge Detection



The waveform confirms that the raw TR_WR signal passes through the Sync function correctly and then after 10 cycles generates the debounced signal. The Tx_WR_d is always exactly one posedge clock behind the debounced version. Lastly, the Tx_WR_rising correctly goes high when the TR_WR_debounce is 1 and Tx_WR_d is 0, as explained earlier in the implementation section.

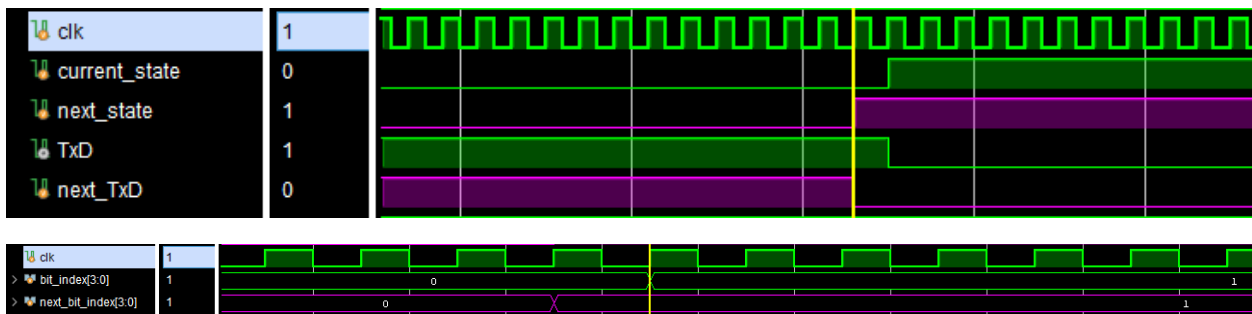
2. Verification of Bit Transmission Timing



TxD transmits one bit every 16 pulses of sample_ENABLE, matching the baud-controller's sampling frequency. When transmission begins, the Tx_BUSY signal and the current state goes high and remains high

3.Verification of “Current” vs “Next” Counter Registers

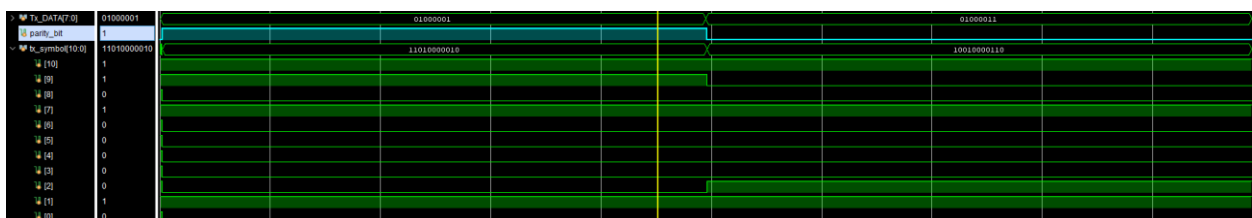
To verify that the FSM pipeline behaves correctly, we check that each “next” signal updates exactly one clock cycle before its corresponding “current” signal.



The signals used where: current_state, TxD and bit_index.

4. Verification of tx_symbol Construction, Parity, and Serial Output.

The Tx_DATA takes the value “A” and “C”



The parity bit correctly is 1 when there is an odd number of ones for “A” and 0 when there is an even number of ones for “C”.

The Tx_symbol stores the Tx_DATA at 1-8 bits the parity bit at the 9th bit, and the start and stop bits at 0 and 10th position.

3.4 Experiment / Final Implementation

During the first week of testing, the UART Transmitter operated correctly on the FPGA with no issues. However, the initial design was not structured as a proper FSM instead, it relied on a single sequential always block that handled all logic at once, resembling more of a software implementation rather than a hardware FSM architecture. In the second week, we remade the design and reorganized it into the clean FSM structure described above, separating next-state logic from registered behavior while keeping the core logic the same. After restructuring, the transmitter continued to work correctly.

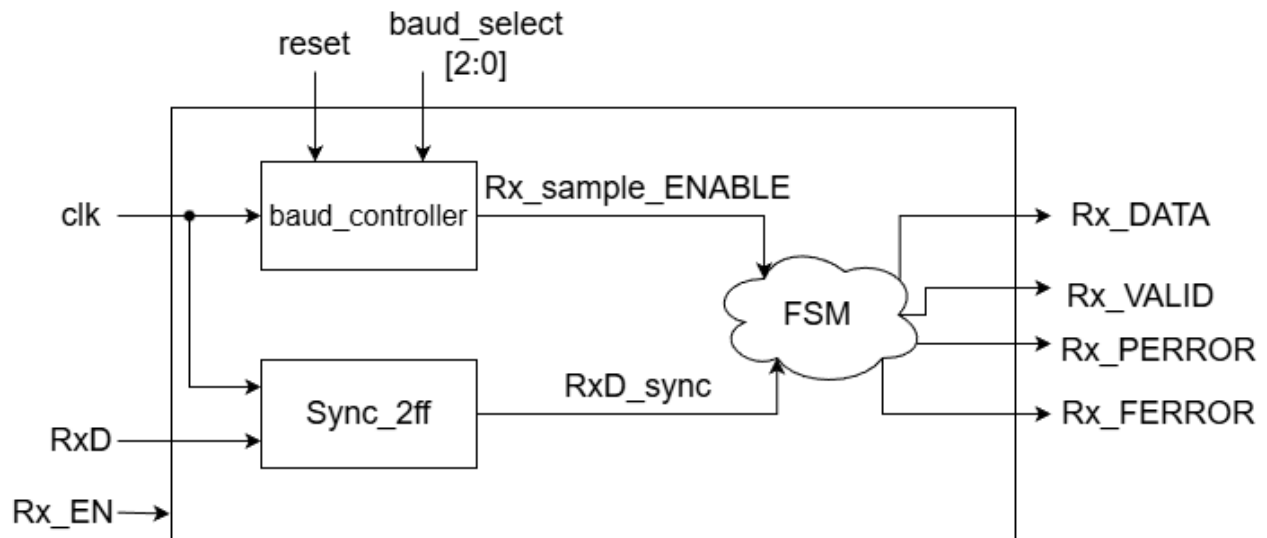
About the warnings in Vivado:

During synthesis, Vivado issued a few warnings related to sensitivity lists and unused registers, but all of them are expected and harmless. Signals such as TxD, Tx_BUSY, and the parity_bit are not included in combinational sensitivity lists and don't need to be, their absence doesn't cause any difference. The message indicating that parity_bit_reg was removed simply reflects optimization: the register was only used as an helper value, so the synthesizer correctly eliminated it. The simulation timescale warning appears only because some modules don't define timescale and which is not necessary, only the testbench must define a timescale. “No constraints selected for write”. It appears because the synthesis step does not actively write out constraint files, and Vivado simply reports that no constraints were selected for output. The project still uses the correct XDC constraints. This warning only indicates that synthesis is not generating new constraint files, not that constraints are missing or ignored.

4. Part C – UART Receiver

4.1 Introduction

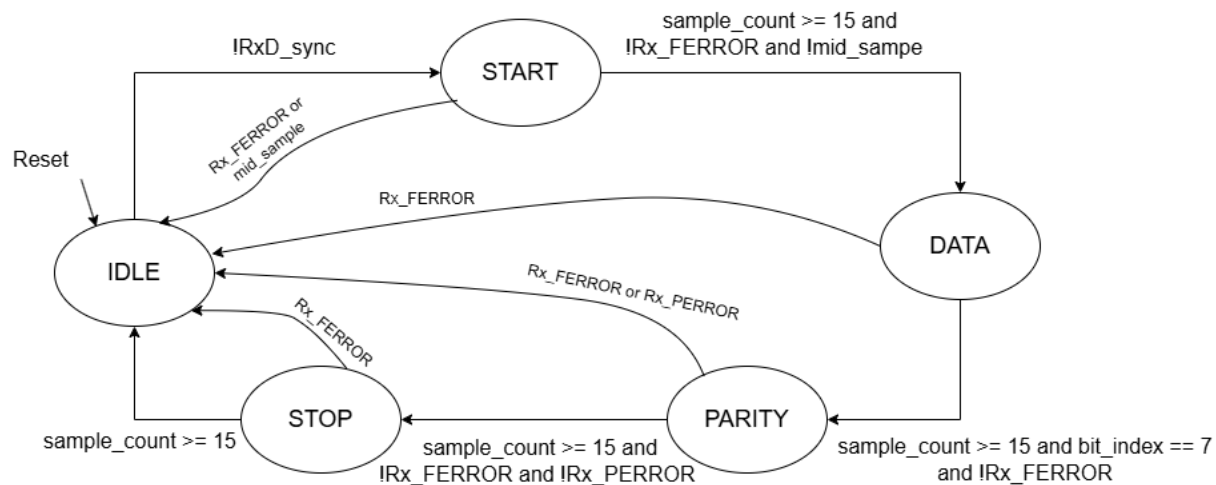
The UART Receiver module is responsible for reading an incoming serial UART stream and interpreting its bits according to the UART protocol. It operates using oversampling, a dedicated baud controller, synchronized inputs, and a structured finite state machine (FSM). As the serial line transitions through the start bit, data bits, parity bit, and stop bit, the receiver samples the incoming signal and processes each part of the frame in sequence. While reading the symbol, the module simultaneously performs checks for noise, parity correctness, and framing consistency, allowing it to determine whether the received byte is valid or corrupted.



4.2 Implementation

4.2.1 Finite State Machine (FSM)

The transmitter operates as a five-state FSM:



IDLE State

In this state:

- Error flags (**Rx_VALID**, **Rx_PERROR**, **Rx_FERROR**) are set to 0.
- Sampling counters are reset.

The receiver monitors the synchronized input “**RxD_sync**” which indicates the beginning of a start bit. When “**RxD_sync**” transitions from 1 to 0, the FSM moves to the **START** state.

START State

Here the receiver performs oversampling of the start bit. It examines the middle samples 7, 8, 9, and 10 (out of 16) to confirm that the signal remains consistently 0. If any mismatch occurs between consecutive samples (detected with a XOR), a framing error is raised, and the FSM returns immediately to the IDLE state.

If all samples agree, the FSM moves to the DATA state.

DATE State

The receiver processes all eight data bits, with the least significant bit first.

For each bit:

- Sixteen sample_ENABLE pulses define the bit period.
- Samples 7–10 are compared to check if any of them takes a different value.
- If their value stays consistent, the bit value is preserved at sample 10.
- If “bit_index” reaches 7 after latching, the FSM transitions to the PARITY state. In the case of a mismatch the “Rx_FERROR” triggers and the FSM resets to the IDLE state

PARITY State

This state oversamples the parity bit. After validating sample consistency again with the middle bits, the received parity bit is compared with the calculated paritybit that the receiver computes. If they differ, a “Rx_PERROR” is flagged and the FSM resets to the IDLE state. Otherwise, the FSM proceeds to the STOP state.

STOP State

The final bit “stop bit” must be 1. Again samples 7–10 must match and also have the value 1.

If they do:

- The received byte is presented on Rx_DATA.
- Rx_VALID pulses for exactly one clock cycle.
- The FSM returns to IDLE.

If not, a “Rx_FERROR” occurs and the byte is discarded.

4.2.2 Modules

Receiver depends on two key supporting modules:

1. Baud Controller
2. Sync_2ff

Both were explained previously. This time, the signal that is being passed through the Sync_2ff is RxD.

4.2.3 Always blocks

The receiver same as the transmitter uses the two main always blocks.

1. Sequential Always Block

Triggered on the rising edge of the system clock or reset.

This block updates all state-holding register:

- `current_state`
- `sample_count`
- `bit_index`
- `Rx_DATA_internal` (shift buffer)
- `prev_sample`
- `Rx_PERROR`, `Rx_FERROR`
- `Rx_VALID`

It also handles reset behavior by clearing all registers and returning the FSM to IDLE.

2. Combinational Always Block

This block evaluates the receiver's current conditions and determines how the FSM should progress as each bit of the incoming UART frame is sampled. Its decisions depend on:

- the current FSM state (IDLE, START, DATA, PARITY, STOP)
- the synchronized input value (`RxD_sync`)
- the oversampling timing provided by `sample_ENABLE`
- the current oversampling counter (`sample_count`)
- the current bit index for the data bits
- the stability of recent samples
- the expected parity computed from the received bits so far
- any error conditions currently developing

Based on these inputs, the combinational block computes every "next" value required by the receiver, including:

- `next_state`, deciding when to transition between IDLE → START → DATA → PARITY → STOP, or when to return to IDLE due to an error

- next_sample_count, incrementing the oversampling counter and resetting it after 15
- next_bit_index, advancing to the next data bit once the current bit's 16 samples are complete
- next_data, inserting each data bit into the internal shift register,
- next_prev_sample, storing the most recent sample to compare it with the next one for noise detection
- next_mid_sample, sorting the value that was determined by the 7-10 bits.
- next_Rx_VALID, determining whether the symbol was correct and whether the Rx_DATA should take the value of shift register data
- next_Rx_PERROR, flagging incorrect parity bit
- next_Rx_FERROR, detecting noise by checking whether samples 7–10 match

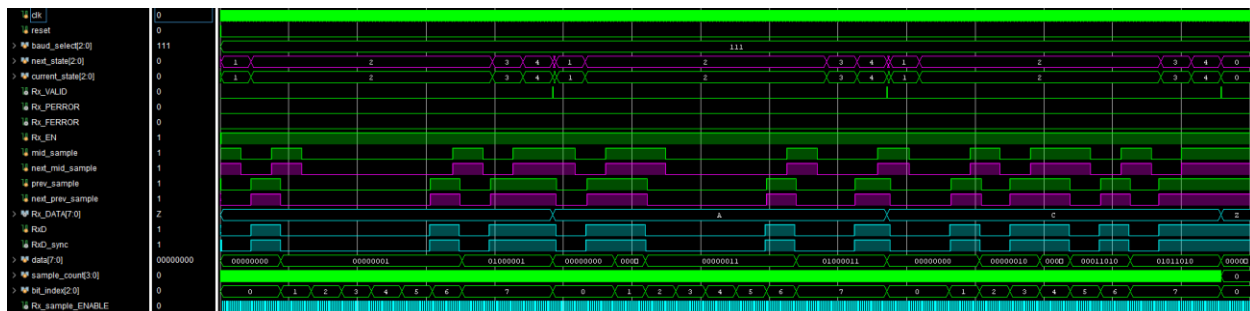
This block thus determines how the receiver should behave in the next clock cycle: whether to continue reading the current bit, move on to the next bit, store a data value, detect an error, or complete the frame. It does all computations, while the sequential block simply stores these results on the next rising clock edge.

4.3 Verification

As before, the verification process for the UART Receiver focuses on confirming that all signals, internal state transitions, counters, and output waveforms correctly follow the UART Receiver protocol.

The testbench that acts as the transmitter sends 3 symbols: “A”, “C” and “Z”.

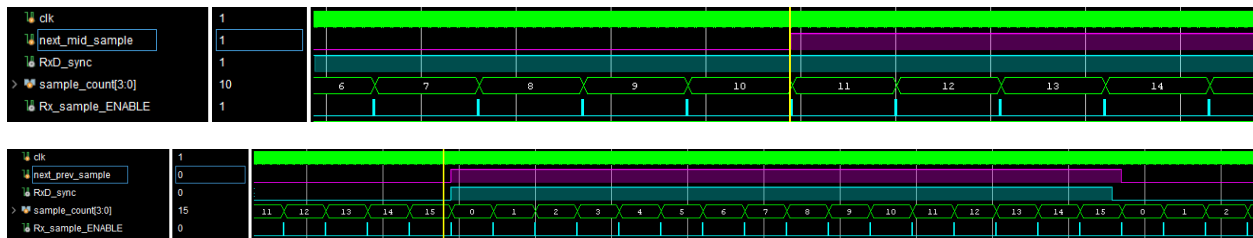
To achieve this, simulation is inspected in several focused segments. Again, the signals with pink, are the “next” state of the signals above them. Not all “next” state signals were added to make the waveform less chaotic.



The primary objectives of this verification are as follows:

1. Verification of TR_WR Synchronization, Debouncing, and Edge Detection

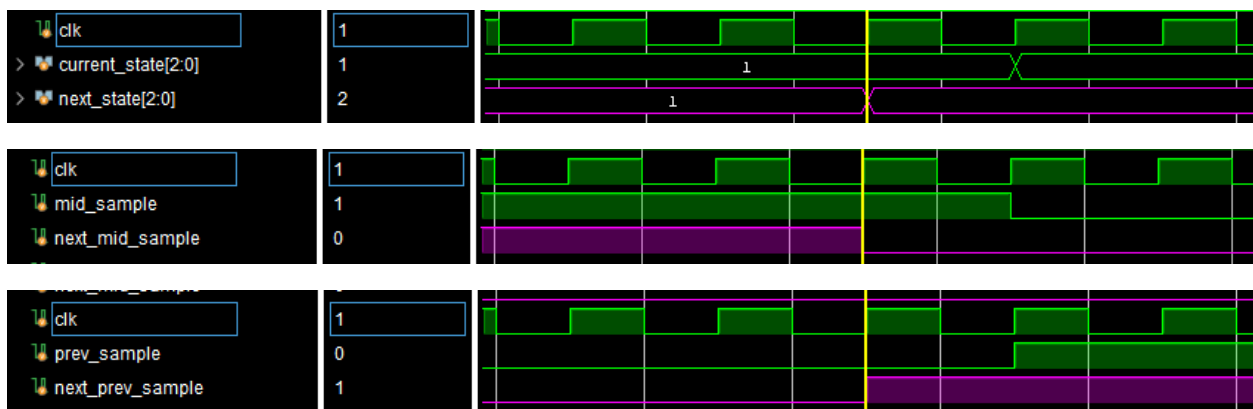
2. Verification of Bit Received Timing



RxD delivers from the transistor one bit every 16 pulses of `sample_ENABLE`, matching the baud-controller's sampling frequency. The `next_mid_sample` correctly takes the value of `RxD_sync` on every 10th `sample_ENABLE`. Also, the `next_prev_sample` correctly takes the value of `RxD_sync` at every `sample_ENABLE` peak.

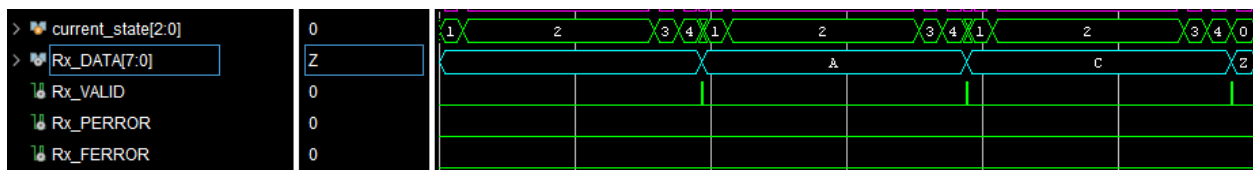
3. Verification of “Current” vs “Next” Counter Registers

Just like the Transmitter, to verify that the FSM pipeline behaves correctly, we check that each “next” signal updates exactly one clock cycle before its corresponding “current” signal.



The signals used whereagain `current_state`, `mid_sample` and `prev_sample`.

4. Verification of Rx_DATA.



`Rx_DATA` correctly takes the values of the symbols that were sent “A”, “C” and “Z” only after `Rx_VALID` signal goes high to indicate that the symbol was received without any errors. Obviously, `Rx_FERROR` and `Rx_ERROR` never go high, resulting to the rising of `Rx_VALID`.

5. Rx_FERROR, Rx_ERROR

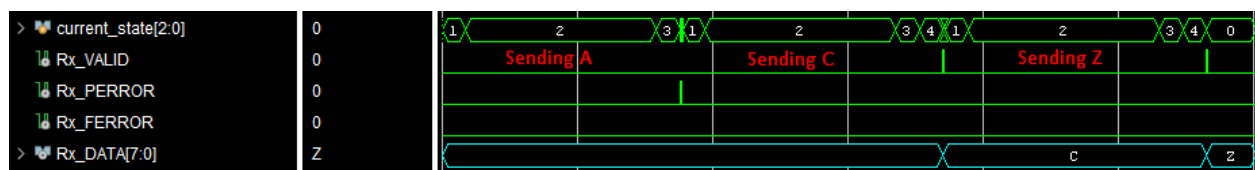
In the testbench there are 3 tasks. Since now only the first task was used, that sends the symbols with correct start, parity and stop bits. The other 2 tasks have been done to test the FERROR and PERROR signals.

The first one sends a symbol with a stop bit 0. The following waveform will send the “A” and “Z” as previously, but the “C” will be sent with faulty stop bit:



`Rx_DATA` takes the value of A after it was sent and the `Rx_VALID` = 1. The FERROR goes high at the end of the current_state = 4 = STOP state indicating that the stop bit when sending C is wrong, so the `Rx_DATA` doesn't take the value of C and stays as A. Then the Z was sent successfully as well as A, so at the end the `Rx_DATA` can be seen taking the value Z.

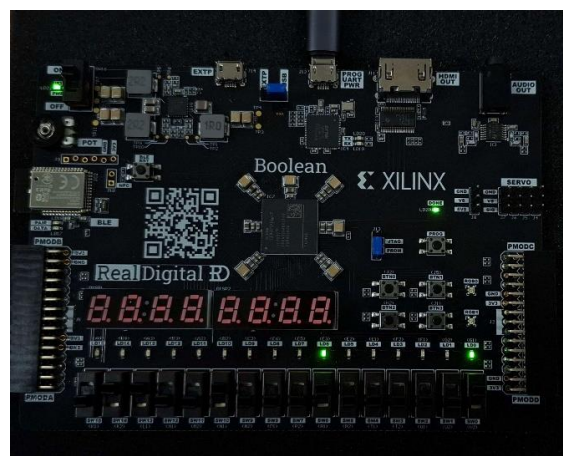
Next, we send A with a wrong parity bit and C, correctly:



In the same fashion as with the wrong stop bit, the `Rx_DATA` only takes the values of C and Z after `Rx_VALID` was triggered. `Rx_PERROR` is triggered at current_state = 3 = PARITY state and then the current state goes back to IDLE to start sending the C symbol.

4.4 Experiment / Final Implementation

The picture below shows “A” being received and the corresponding LED being lit up.

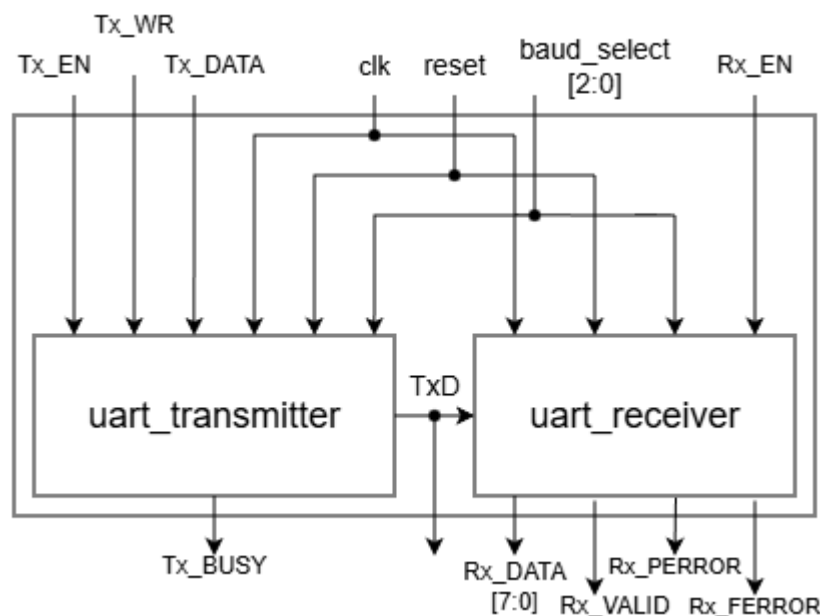


About the warnings in Vivado: Again we have the timescale warning and the constraints for write warning. Both were addressed in Part B

5. Part D – Transmitter-Receiver Serial Data Transfer Connection

5.1 Introduction

Part D brings together the UART Transmitter and UART Receiver developed in the previous sections into a single integrated communication system. The objective of this part is to verify end-to-end UART communication: a byte written to the transmitter should propagate through the serial line and be correctly interpreted by the receiver. Because the transmitter and receiver already implement all UART protocol mechanisms internally (start bit, eight data bits, even parity, stop bit, oversampling, and error checks), this part focuses solely on connecting the two modules into a functional top-level system rather than redefining their internal logic.



5.2 Implementation

Only two modules are instantiated in the top-level module: transmitter and receiver. The other modules are instantiated in them:

- Transmitter
 - Baud Controller
 - Sync_2ff
 - Debouncer
- Receiver
 - Baud Controller
 - Sync_2ff

In this top-level module:

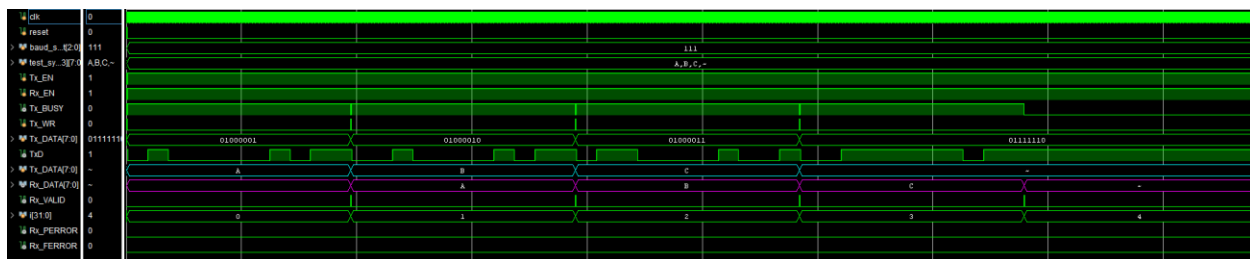
- Clock and reset are forwarded directly to both modules.
- Transmitter receives Tx_DATA, Tx_EN, Tx_WR, and the selected baud_select.
- Transmitter's output pin TxD is wired directly to the receiver's RxD, forming the communication loop.
- Receiver uses the same baud rate selection to remain synchronized and receives the Rx_EN.
- Top module exposes all receiver outputs Rx_DATA, Rx_VALID, Rx_ERROR, Rx_FERROR as well as the transmitter status Tx_BUSY.

Since this part of the design is purely structural, the top-level module contains no always blocks

5.3 Verification

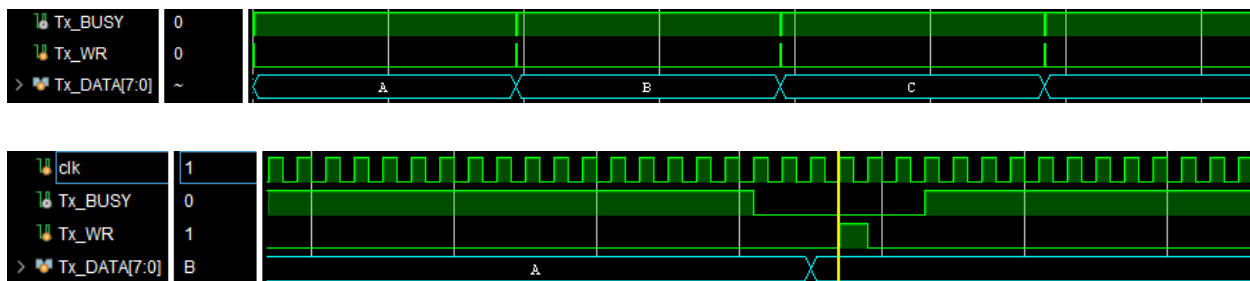
The testbench doesn't use any time delays, just "wait" commands. It waits for Tx_BUSY to become zero and then calls a task that waits for the next posedge of the clock, to change the value of Tx_WR to 1. Tx_WR stays high for only one pulse.

Tx_DATA was added 2 times, one in binary form (green) to help with following the values of the TxD and one in ASCII form (aqua) to clearly show what symbol is being sent at each point.



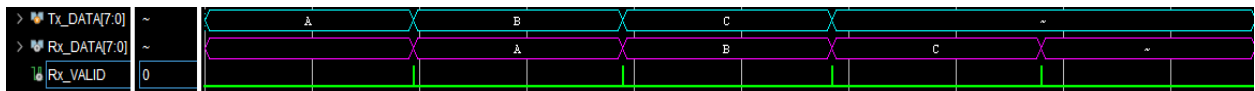
The primary objectives of this verification are as follows:

1. Tx_BUSY and Tx_WR synchronization



In the 1st waveform above, Tx_WR can be seen rising only after Tx_BUSY is zero meaning the transmitter finished sending a symbol and is waiting to be told to send a next one. The 2nd waveform shows a closer look to the signals.

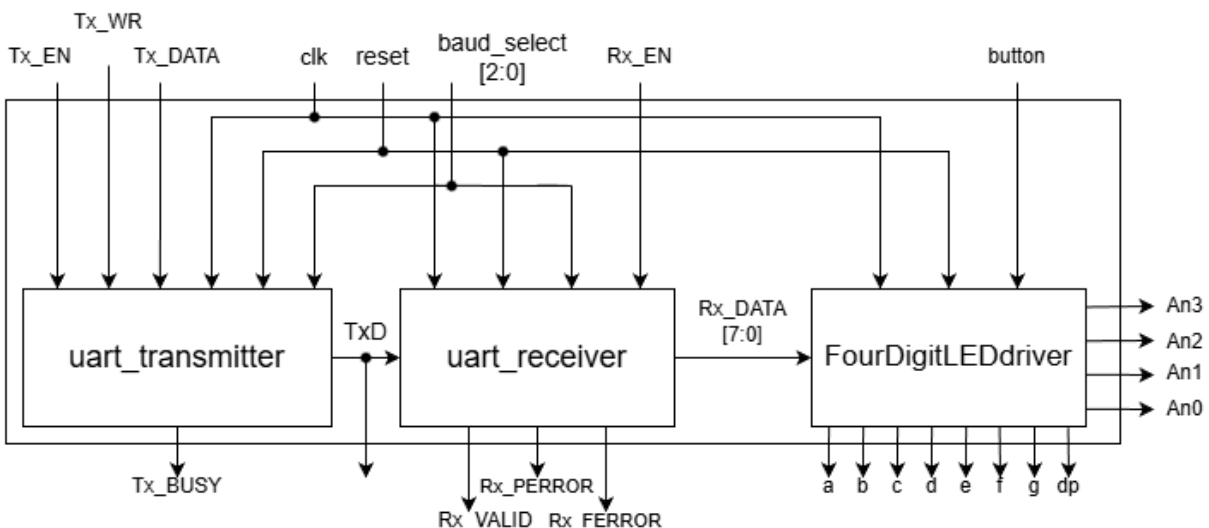
2. Tx_DATA passing to Rx_DATA



6. Part D EXTRA – UART Integration with Four-Digit LED Display

6.1 Introduction

In this extended version of Part D, the top-level design incorporates not only the UART Transmitter and UART Receiver but also the FourDigitLEDdriver from Lab 1. The goal is to visually display each successfully received UART byte on the 7-segment LED display. Unlike the earlier parts, this section does not introduce new UART logic, instead it integrates the existing communication path with a visual output interface.



6.1 Implementation

FourDigitLEDdriver internally generates its own newclk for LED multiplexing, used exclusively by its submodules and unrelated to the UART timing.

The only new logic that must be described is the modified Anode_driver, which now interprets the received UART byte and places it into one of the four display positions.

FourDigitLEDdriver receives the Rx_DATA from the UART receiver with a new port named “data”. This “data” is then passed to the Anode_driver. The new Anode_driver receives:

- Clock and Reset

- The debounced button signal
- The internal display counter
- The data byte from the UART

Its purpose is to convert the received byte into a hexadecimal number and store it into one of the four display “slots” (message[0] to message[3]), depending on how many times the button has been pressed.

There are 3 always blocks in Anode_driver:

1. Button-Controlled Rotation Storage

Each valid button press stores the newly received hex digit into one of four display positions. The positions advance cyclically (0→1→2→3→0) as the user continues pressing the button. Reset clears all digits and restarts the rotation from the first position.

2. Character Mapping (ASCII → Hex Value)

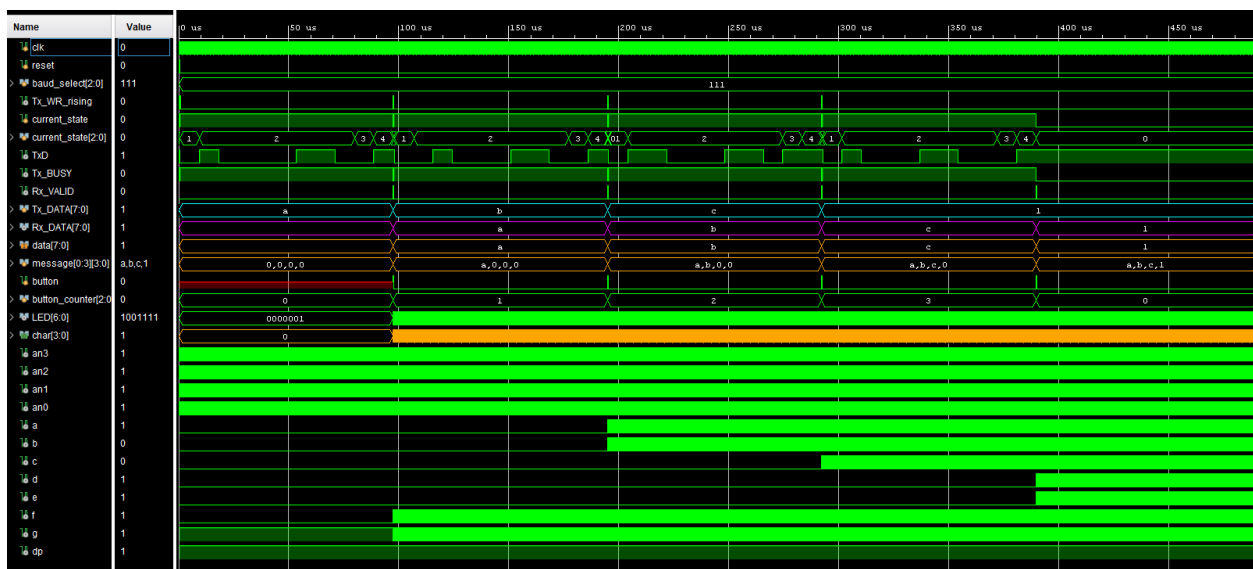
The always block converts the received Rx_DATA into a hexadecimal digit based on its ASCII value. Only characters 0–9 and a–f are recognized, everything else defaults to zero. This converted value is then prepared for potential display into a register “next_value”.

3. Digit Activation and Character Output

The display cycles rapidly through the four digits, activating one at a time. For each active digit, the corresponding stored hex value is presented as the output character.

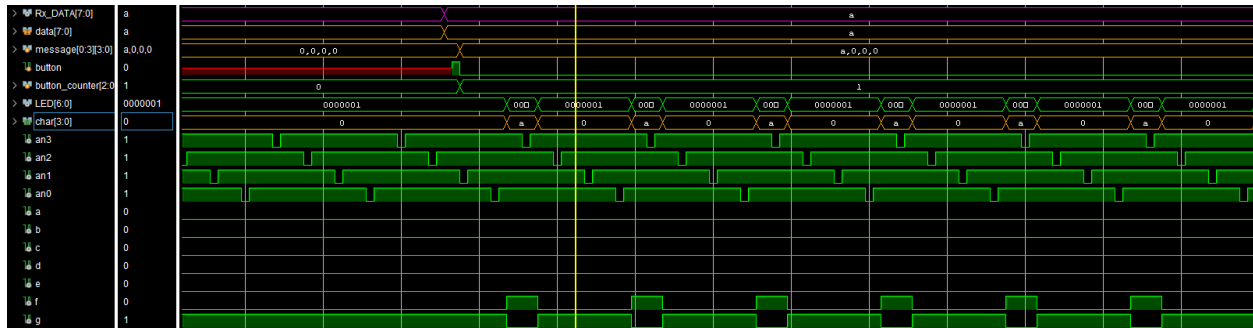
6.3 Verification

The transmitter in the testbench sends 4 symbols: “A”, “B” “C” and “1”.

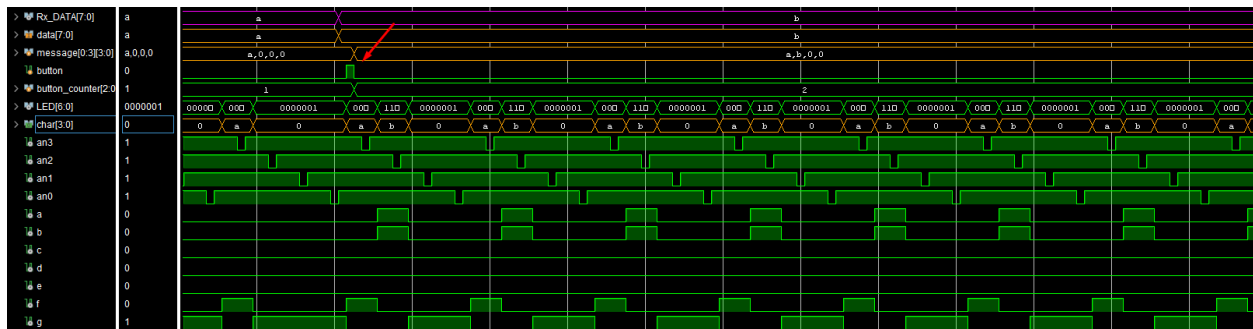


The following 3 waveforms will show a close up of the “char”, that is the input in the LEDdecoder, the module that gives the correct value to the cathodes “a-g”. At the start the value of “char” is zero, meaning that the 4 displays show the “0”.

After the symbol “a” was successfully send, the “char” takes its only when around an3 turn. This way the first display is “a” and all the others are still “0”

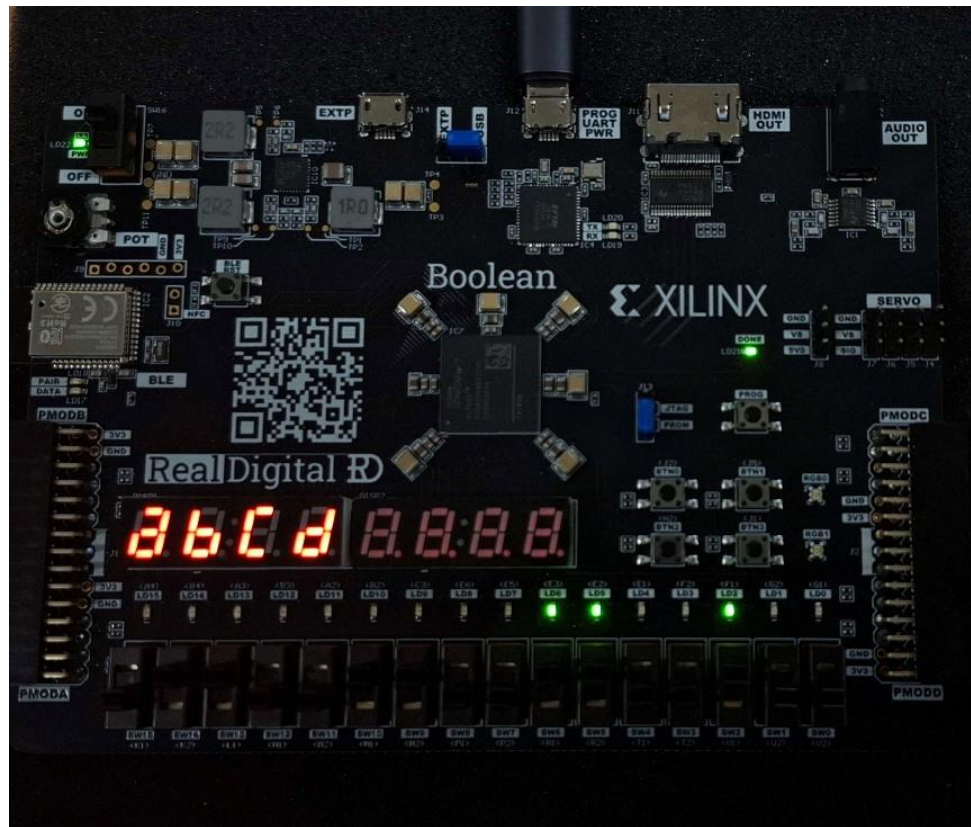


The following waveform pictures the value “b” that takes place around an2 and then finally, the last one depicts all four symbols being sent around their corresponding anode each.



6.4 Experiment / Final Implementation

The following picture shows the symbols “a”, “b”, “c” and “d” being lit on the 7 segment-display confirming the later waveform.



About the warnings in Vivado:

During synthesis and simulation a few warnings appeared, but none of them affect the correctness or functionality of the design. The missing sensitivity-list warning refers to a signal that is updated only inside a sequential always block, so it is harmless. The unused-register warning simply indicates that an internal parity register was optimized away because the transmitter uses the computed parity directly. The MMCME2 warning appears because the LED driver's clocking block is instantiated with only the necessary ports connected, which is valid for this project. The constant-driven dp output is intentional since the decimal point is always off. Finally, the simulation timescale warning appears only because some modules define a timescale and others do not; this does not affect behavior because the baud controller logic is driven entirely by the clock, not by simulation delays. Overall, these warnings are expected and do not indicate any functional issues.